計算機構成論 第14回
―パイプライン処理―

大連理工大学・立命館大学 国際情報ソフトウェア学部

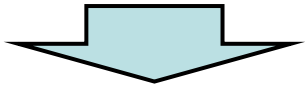
双見京介

講義内容

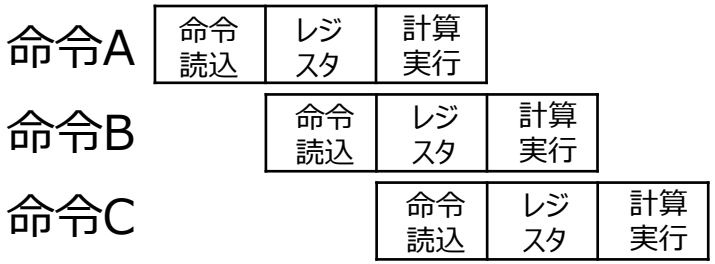
- パイプライン処理
  - 概説
  - データパスの構築
  - パイプライン処理の実行
  - パイプライン処理の問題点

さらなる性能の向上

- 計算機の性能の決定要因
  - 1. 単位時間あたりの実行命令数
  - 2. クロック・サイクル時間
  - 3. CPI (命令あたりのクロック・サイクル数)
- 単位時間あたりの実行命令数を増やせないか？



- パイプライン処理

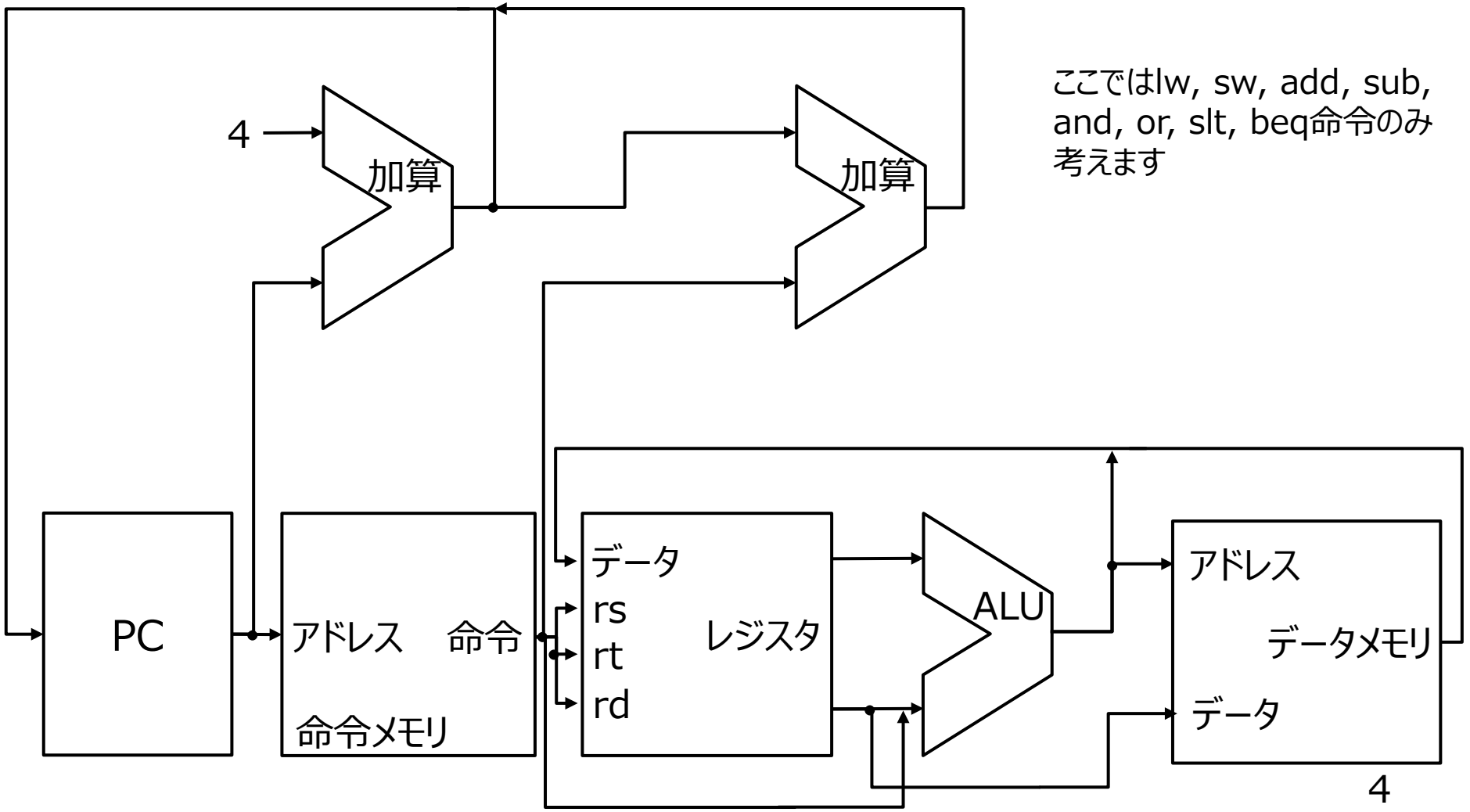


命令の実行

- 命令の実行段階
  - メモリから命令を取り出す
    ＝フェッチする (fetch)
    - 命令の内容をレジスタに転送
    - PC(プログラムカウンタ)を4進める
  - オペランドのレジスタの値を読み出す

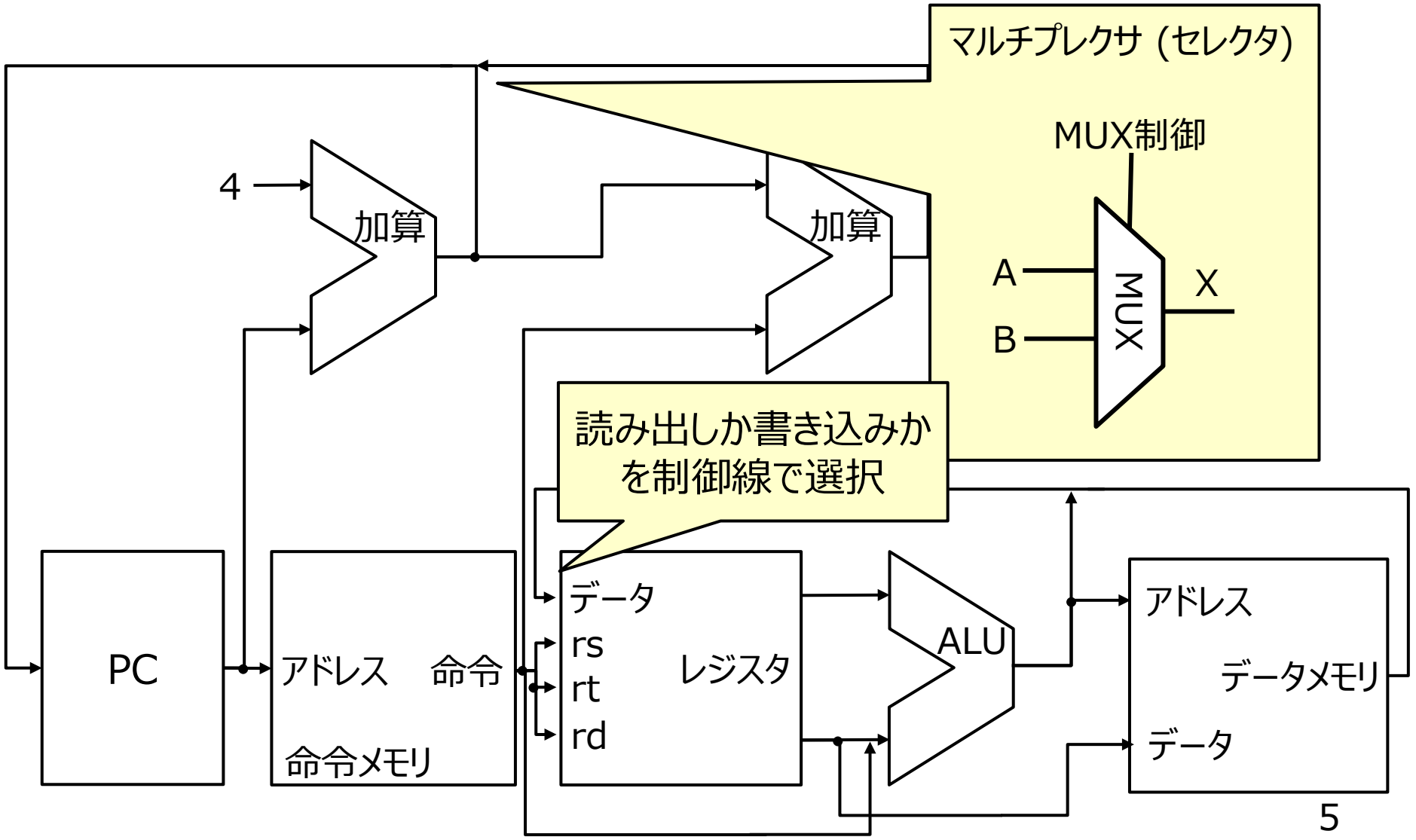
  - その後、命令ごとに異なる処理へ
    - 処理内容は結構似ている
      - 算術論理演算の実行 ＝ ALU使用
      - lw, sw → アドレス値加算 ＝ ALU使用

MIPS命令実装方式の概念図



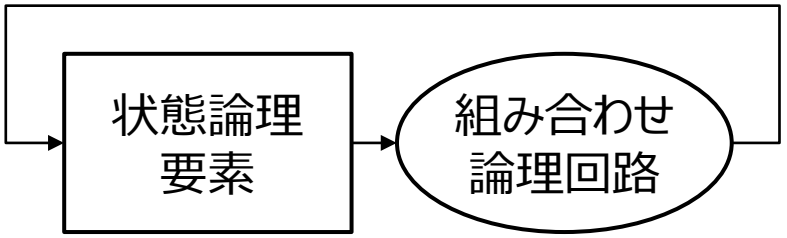
ここではlw, sw, add, sub, and, or, slt, beq命令のみ考えます

MIPS命令実装方式の概念図

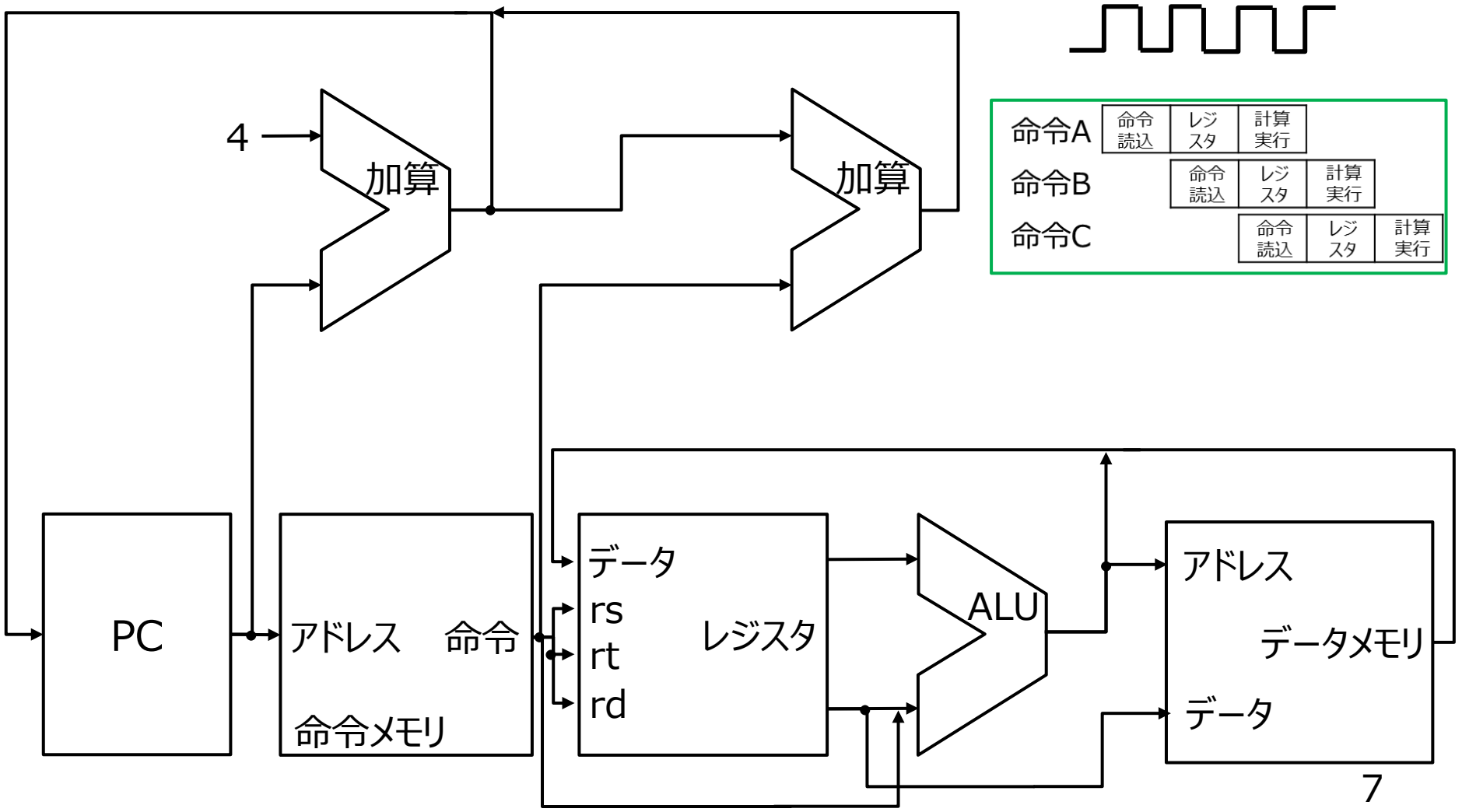


論理設計とクロック方式

- 計算機設計の際は、
  クロックの刻み方を決めないといけない
  - 組み合わせ論理要素
    - 内部状態を持たない (ALUなど)
  - 状態論理要素
    - 内部状態を持つ (レジスタやメモリなど)
    - クロック・パルスごとに状態が進む
- エッジ・トリガ・クロック方式
  - クロック・パルスの
    高低切り替わりの際に
    状態を進める



MIPS命令実装方式の概念図



確認問題

- 次の説明に該当する用語を答えよ。
  - 複数の命令を時間的にずらして、
    並行的に実行する処理方式
  - メモリから命令を取り出すこと
  - クロック・パルスの高低切り替わりの際に
    状態論理要素の状態を進める方式
  - 制御信号により、複数の入力の中から
    1つを選んで出力する機構

確認問題

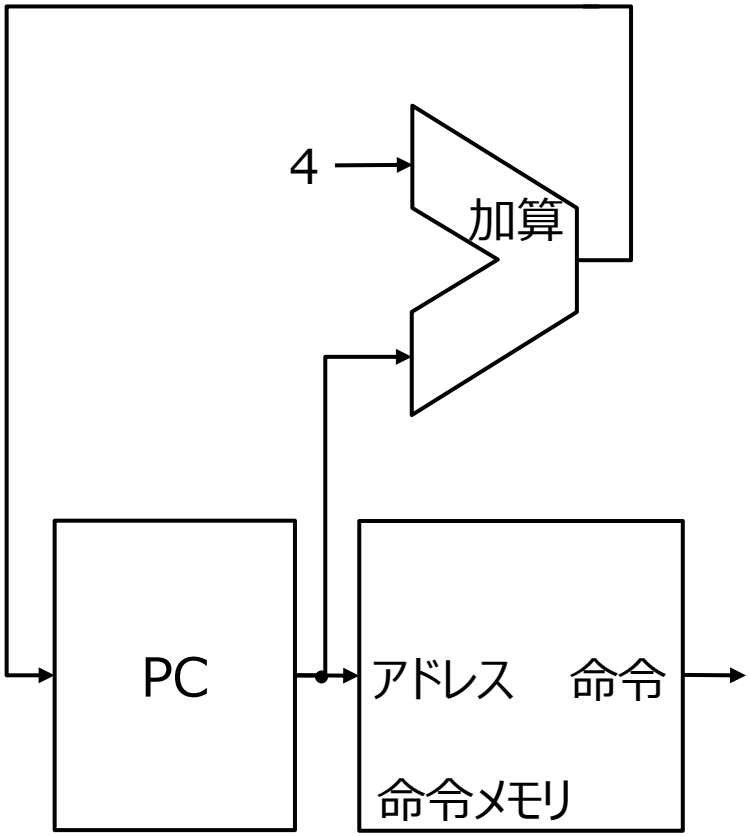
- 次の説明に該当する用語を答えよ。
  - 複数の命令を時間的にずらして、
    並行的に実行する処理方式
  - メモリから命令を取り出すこと
  - クロック・パルスの高低切り替わりの際に
    状態論理要素の状態を進める方式
  - 制御信号により、複数の入力の中から
    1つを選んで出力する機構

講義内容

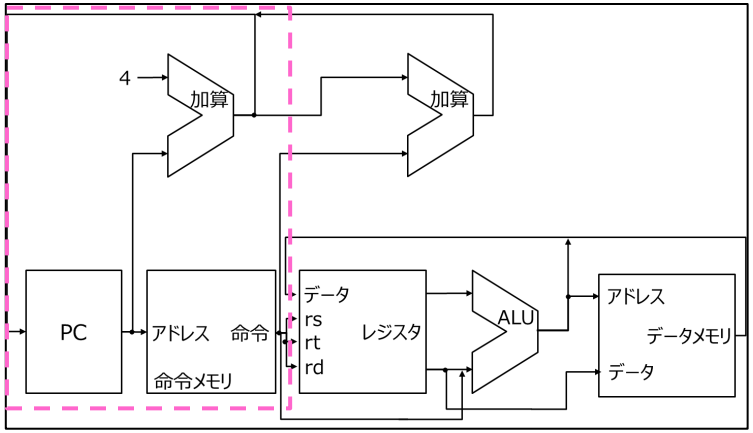
- パイプライン処理
  - 概説
  - データパスの構築
  - パイプライン処理の実行
  - パイプライン処理の問題点

データパスの構築 命令のフェッチ

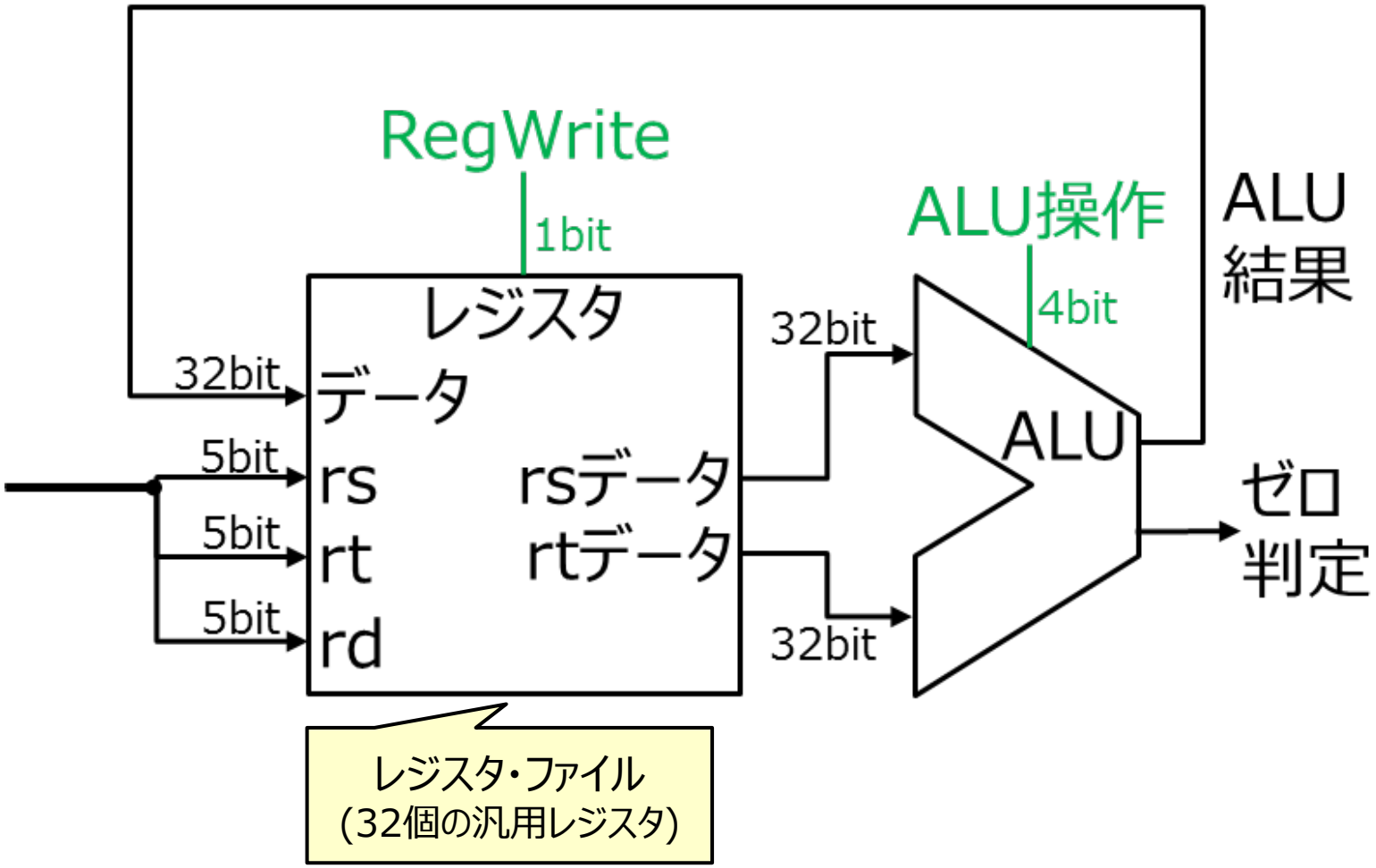
- データパス(datapath)
  - データの経路



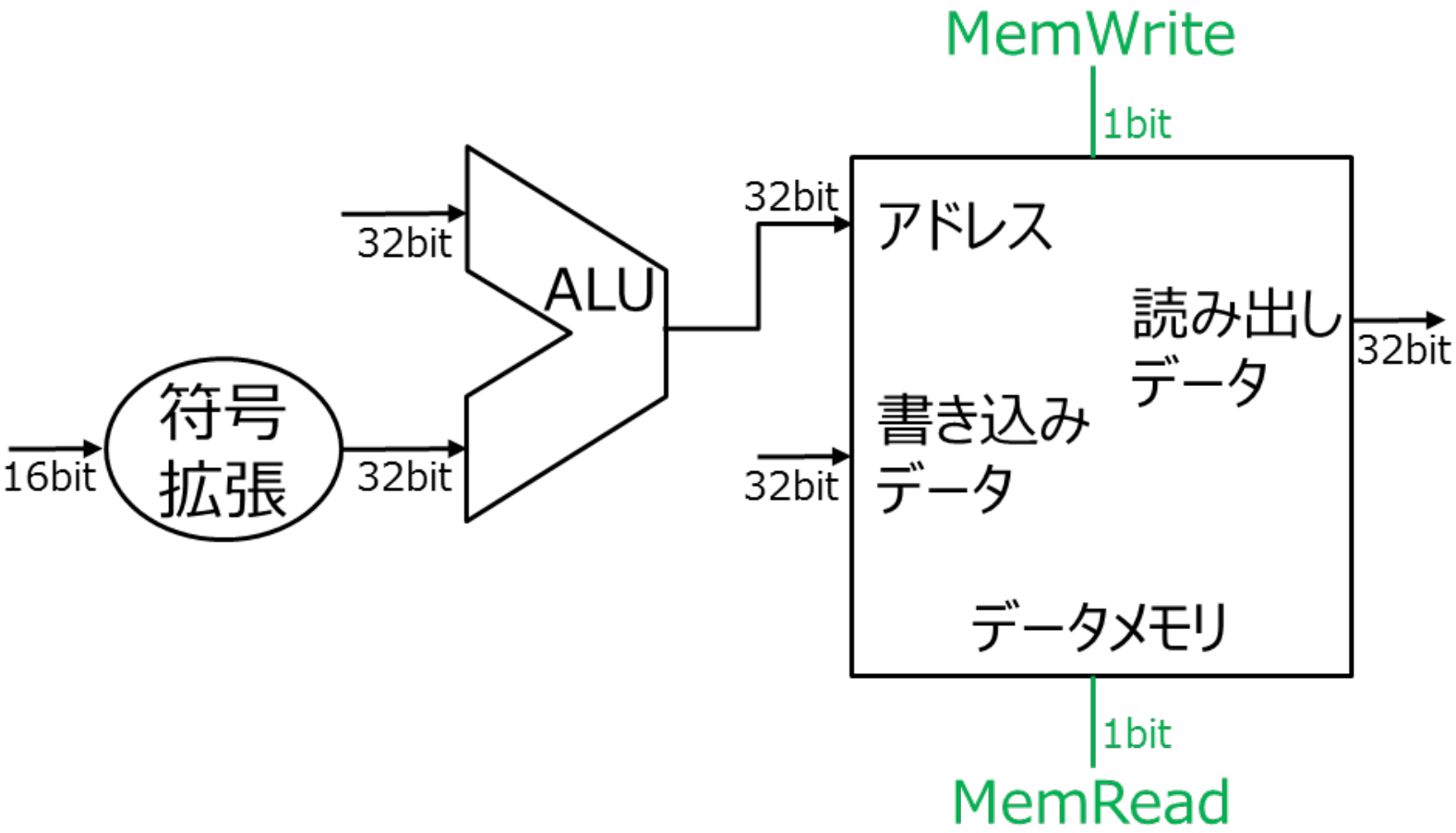
命令をフェッチして
PCを進める部分の
データパス



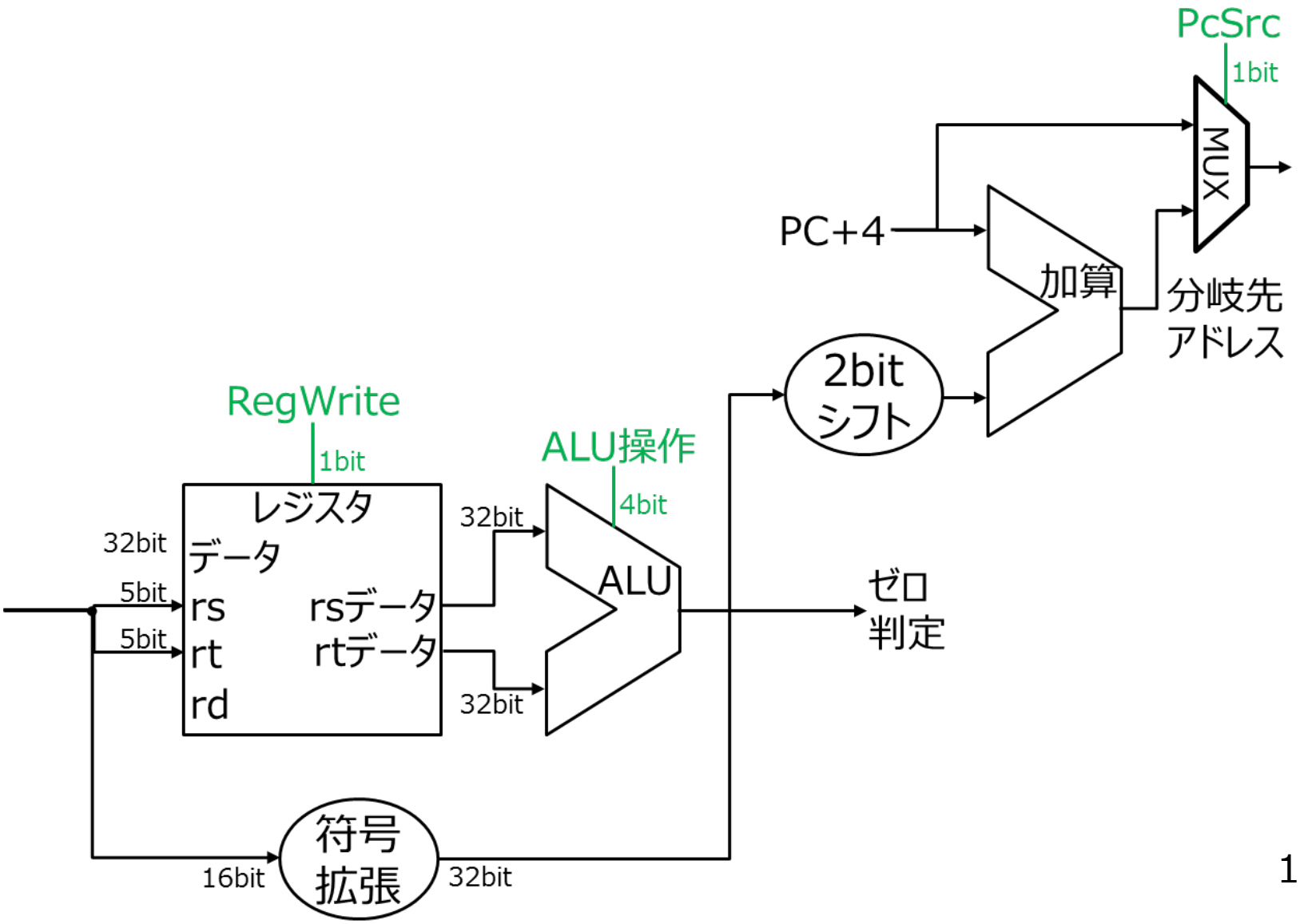
データパスの構築 R形式の命令実行



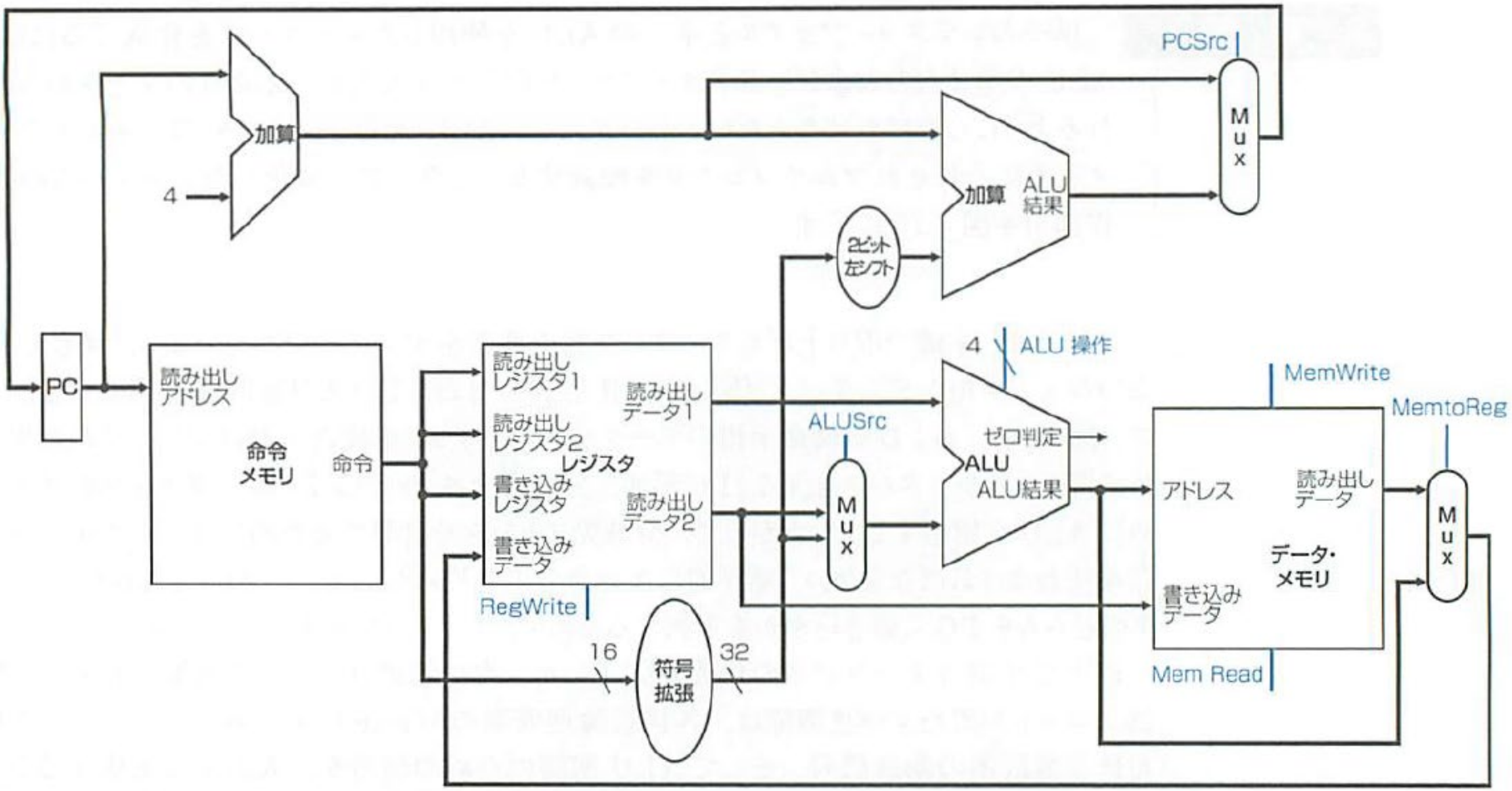
データパスの構築 lw/sw命令の実行



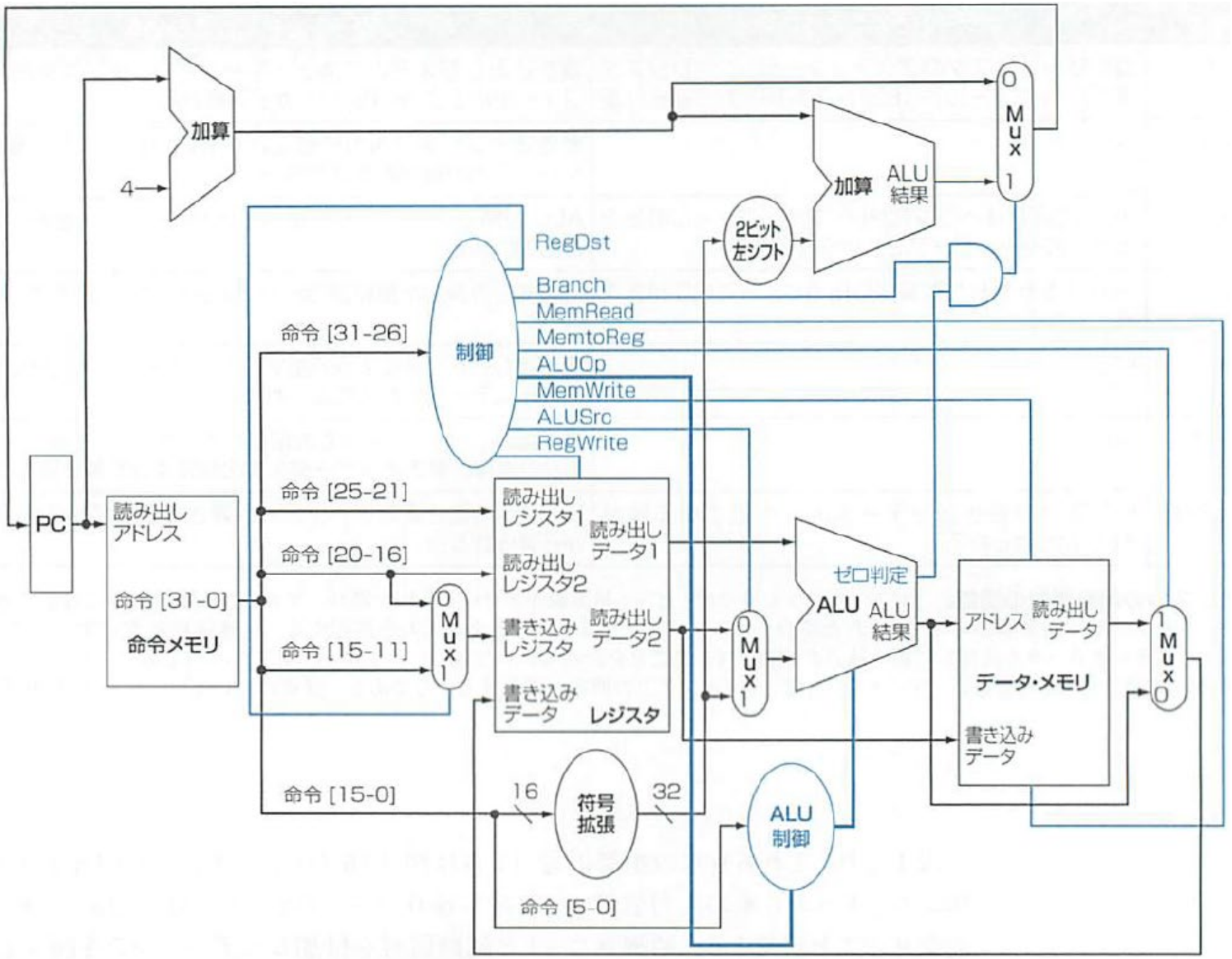
データパスの構築 分岐命令の実行



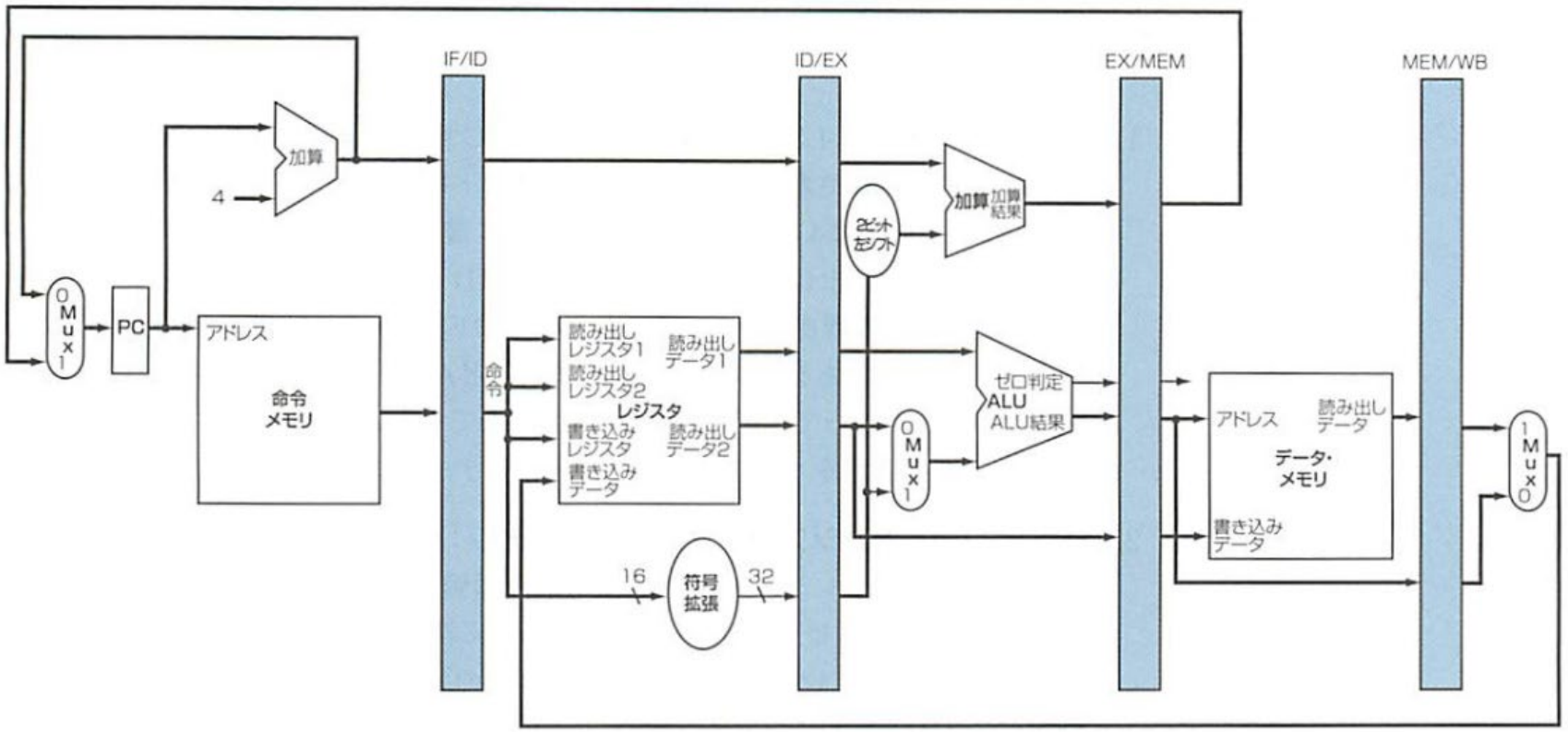
データパスの構築 各部の統合



データパスの構築 さらに詳細化



データパス (パイプライン・レジスタ有)



IF: 命令フェッチ
ID: 命令デコードとレジスタフェッチ
EX: 命令実行／アドレス生成
MEM: データメモリアクセス
WB: 書き込み

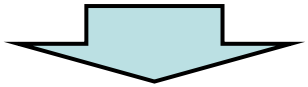
パイプライン・ステージごとに
パイプライン・レジスタが設置される

講義内容

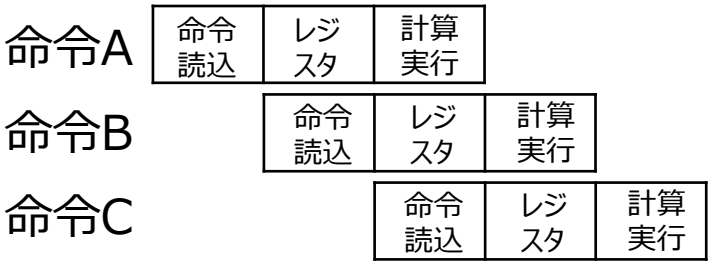
- パイプライン処理
  - 概説
  - データパスの構築
  - パイプライン処理の実行
  - パイプライン処理の問題点

さらなる性能の向上　再掲

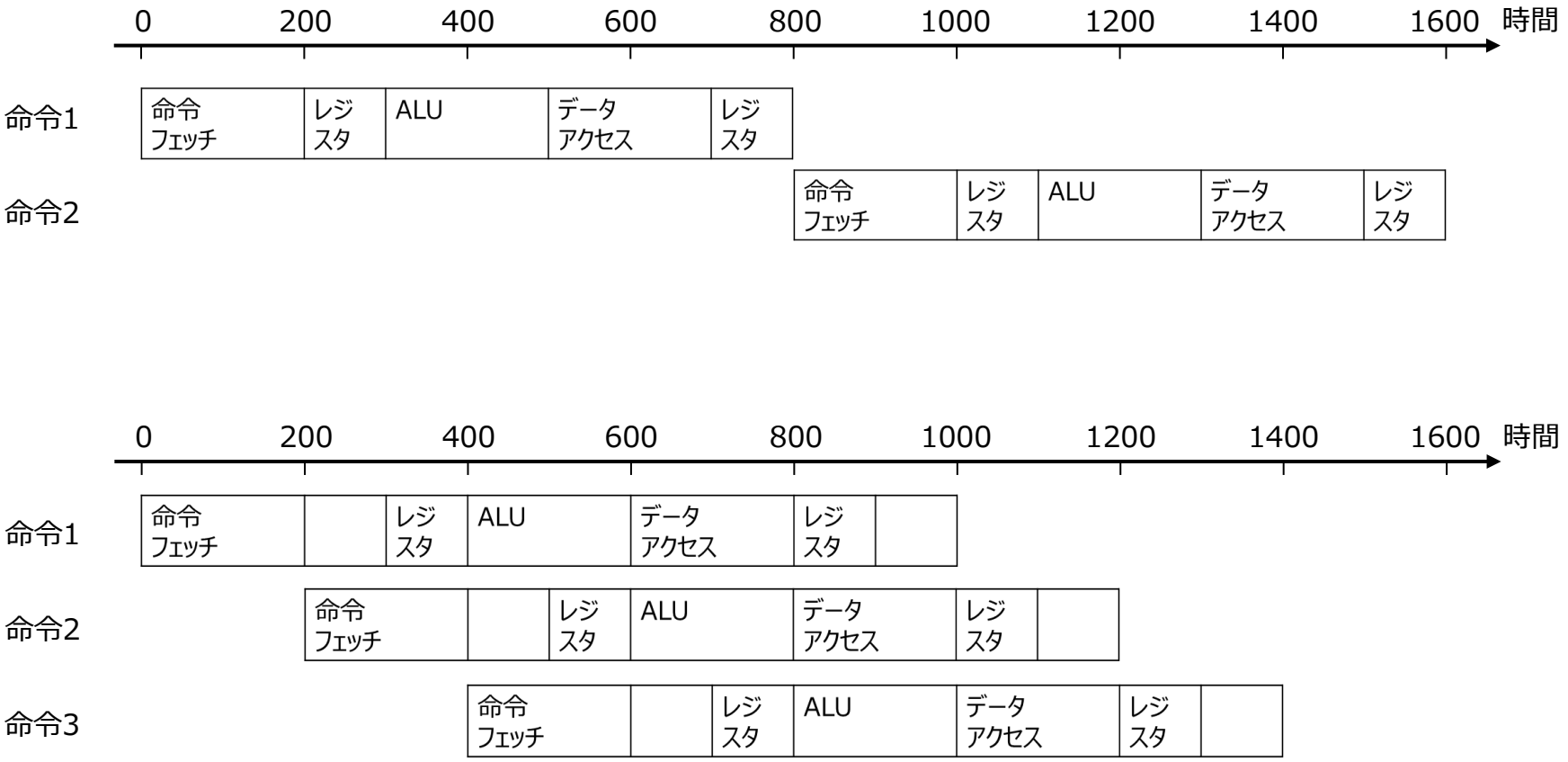
- 計算機の性能の決定要因
  - 1. 単位時間あたりの実行命令数
  - 2. クロック・サイクル時間
  - 3. CPI (命令あたりのクロック・サイクル数)
- 単位時間あたりの実行命令数を増やせないか？



- パイプライン処理



パイプライン処理の実行イメージ



パイプラインは個々の命令を完了させるのにかかる時間(レイテンシ, latency)を短縮するわけではない

講義内容

- パイプライン処理
  - 概説
  - データパスの構築
  - パイプライン処理の実行
  - パイプライン処理の問題点

パイプライン処理の問題点

- データ・ハザード　data hazards
  - 命令の実行に必要なデータが
    まだ利用可能でないため、
    予定している命令を実行できないこと

```
add $s0, $t0, $t1
sub $t2, $s0, $t3
```

フォワーディング（バイパシング）forwarding (bypassing)：
計算結果をすぐに使えるようにする

コードの並べ替え：

コードの実行順を変えることでストールを回避

パイプライン・ストール：　stall[V/N]行き詰まること

実行を待機して、計算結果が来るのを待つ
(NOP命令 (何もしない命令) の導入)
→ 実行が遅くなるので、できるだけ避ける

パイプライン処理の問題点

- 構造ハザード structural hazards
  - パイプライン処理において、
    命令の実行に必要なハードウェアが
    競合するため、予定された命令を
    実行できないこと

    e.g., 複数の命令で同時にメモリの読み書き

    競合する命令を遅らせるなどして対処

パイプライン処理の問題点

- 制御ハザード control hazards
  - 条件分岐が存在するため、命令を実行するべきかどうか判断できず、所定のクロック・サイクル内で命令を実行できないこと
  - 分岐ハザードともいう

予測：
分岐の結果を予測(例えば、常に条件不成立)して命令の実行を進めておく

確認問題

- 以下の各文は正しいか。○か×で答えよ。
  - パイプライン処理では、命令を1つずつ、直列的に実行する。
  - パイプライン処理によって、各命令の実行に必要な時間を減らすことができる。
  - パイプライン処理は、スループットの向上を目標としている。
  - パイプライン処理において、必要なデータがまだ利用可能でないため、命令を実行できないことを分岐ハザードという。

確認問題

- 以下の各文は正しいか。○か×で答えよ。
  - パイプライン処理では、命令を1つずつ、直列的に実行する
  - パイプライン処理によって、各命令の実行に必要な時間を減らすことができる。
  - パイプライン処理は、スループットの向上を目標としている。
  - パイプライン処理において、必要なデータがまだ利用可能でないため、命令を実行できないことを制御ハザードという。

さらなる学習

- 実行に時間がかかる命令（乗除算など）をパイプラインで扱うには？
  - 通常のパイプラインとは別に処理する etc.
- キャッシュの利用
  - 記憶階層の実現
  - 仮想記憶（2次記憶をキャッシュとして使う）
- 並列処理
  - スーパースカラ
  - ハードウェア・マルチスレッディング

参考文献

- コンピュータの構成と設計 上 第5版
  David A.Patterson, John L. Hennessy 著、
  成田光彰 訳、日経BP社
- コンピュータの構成と設計 下 第5版
  David A.Patterson, John L. Hennessy 著、
  成田光彰 訳、日経BP社

- 画像は教科書からのスキャンです。
  転載・頒布を禁止します。